Each filter stage lends itself to a particular use case, thanks to its specific location in the pipeline, with respect to model binding, action execution, and result execution:

- *Authorization filters*—These run first in the pipeline, so they're useful for protecting your APIs and action methods. If an authorization filter deems the request unauthorized, it short-circuits the request, preventing the rest of the filter pipeline (or action) from running.
- *Resource filters*—After authorization, resource filters are the next filters to run in the pipeline. They can also execute at the *end* of the pipeline, in much the same way that middleware components can handle both the incoming request and the outgoing response. Alternatively, resource filters can completely short-circuit the request pipeline and return a response directly.

Thanks to their early position in the pipeline, resource filters can have a variety of uses. You could add metrics to an action method; prevent an action method from executing if an unsupported content type is requested; or, as they run before model binding, control the way model binding works for that request.

- *Action filters*—Action filters run immediately before and after an action method is executed. As model binding has already happened, action filters let you manipulate the arguments to the method—before it executes—or they can short-circuit the action completely and return a different `IActionResult`. Because they also run after the action executes, they can optionally customize an `IActionResult` returned by the action before the action result is executed.